UNIT_4 LAB EXERCISE

NAME: Jeeva

REG NO: 192421334

1. You are given the number of sides on a die (num_sides), the number of dice to throw (num_dice), and a target sum (target). Develop a program that utilizes dynamic programming to solve the Dice Throw Problem.

```python
def dice_ways(num_dice, num_sides, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1

    for d in range(1, num_dice + 1):
        for s in range(1, target + 1):
            for face in range(1, num_sides + 1):
                if s >= face:
                    dp[d][s] += dp[d - 1][s - face]

    return dp[num_dice][target]


print("Test Case 1:", dice_ways(2, 6, 7))
print("Test Case 2:", dice_ways(3, 4, 10))
```

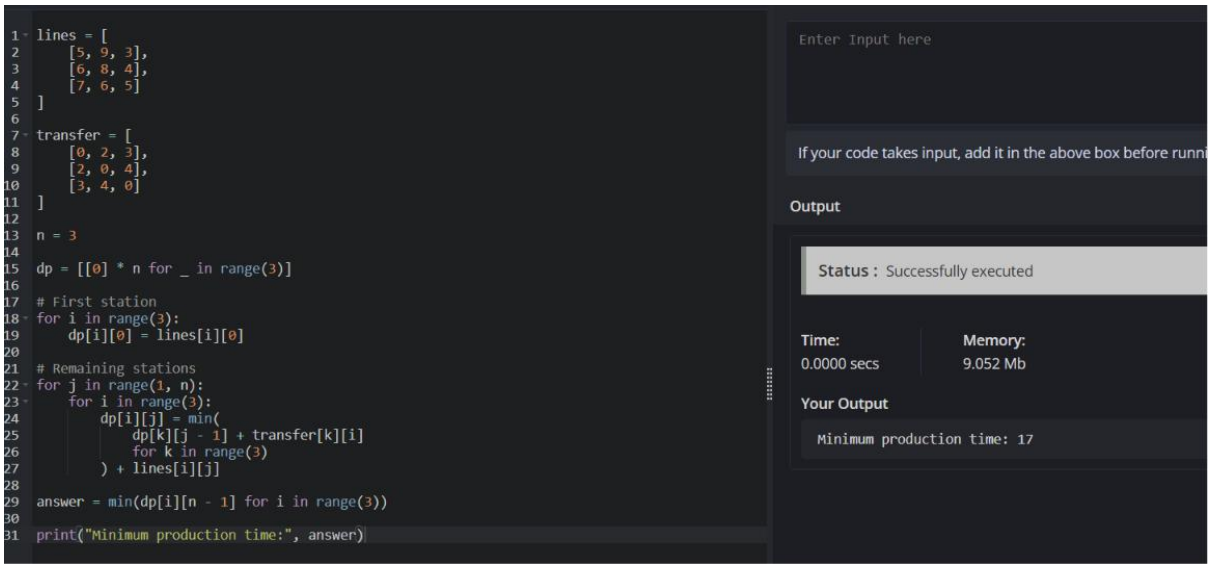
```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.856 Mb

Your Output

Test Case 1: 6
Test Case 2: 6
```

2. In a factory, there are two assembly lines, each with n stations. Each station performs a specific task and takes a certain amount of time to complete. The task must go through each station in order, and there is also a transfer time for switching from one line to another. Given the time taken at each station on both lines and the transfer time between the lines, the goal is to find the minimum time required to process a product from start to end.

```python
def assembly_line(a1, a2, t1, t2, e1, e2, x1, x2):
    n = len(a1)

    dp1 = [0] * n
    dp2 = [0] * n

    dp1[0] = e1 + a1[0]
    dp2[0] = e2 + a2[0]

    for i in range(1, n):
        dp1[i] = min(dp1[i - 1] + a1[i],
                     dp2[i - 1] + t2[i - 1] + a1[i])

        dp2[i] = min(dp2[i - 1] + a2[i],
                     dp1[i - 1] + t1[i - 1] + a2[i])

    return min(dp1[-1] + x1, dp2[-1] + x2)


a1 = [4, 5, 3, 2]
a2 = [2, 10, 1, 4]

t1 = [7, 4, 5]
t2 = [9, 2, 8]

e1, e2 = 10, 12
x1, x2 = 18, 7

print("Minimum time:", assembly_line(a1, a2, t1, t2, e1, e2, x1, x2))
```

```
Ctrl+↵ / ⌘+↵ t here

If your code takes input, add it in the above box before runni

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.976 Mb

Your Output

Minimum time: 35
```

3. An automotive company has three assembly lines (Line 1, Line 2, Line 3) to produce different car models. Each line has a series of stations, and each station takes a certain amount of time to complete its task. Additionally, there are transfer times between lines, and certain dependencies must be respected due to the sequential nature of some tasks. Your goal is to minimize the total production time by determining the optimal scheduling of tasks across these lines, considering the transfer times and dependencies.



```python
lines = [
    [5, 9, 3],
    [6, 8, 4],
    [7, 6, 5]
]

transfer = [
    [0, 2, 3],
    [2, 0, 4],
    [3, 4, 0]
]

n = 3

dp = [[0] * n for _ in range(3)]

# First station
for i in range(3):
    dp[i][0] = lines[i][0]

# Remaining stations
for j in range(1, n):
    for i in range(3):
        dp[i][j] = min(
            dp[k][j - 1] + transfer[k][i]
            for k in range(3)
        ) + lines[i][j]

answer = min(dp[i][n - 1] for i in range(3))

print("Minimum production time:", answer)
```

Enter Input here

If your code takes input, add it in the above box before runni

Output

Status : Successfully executed

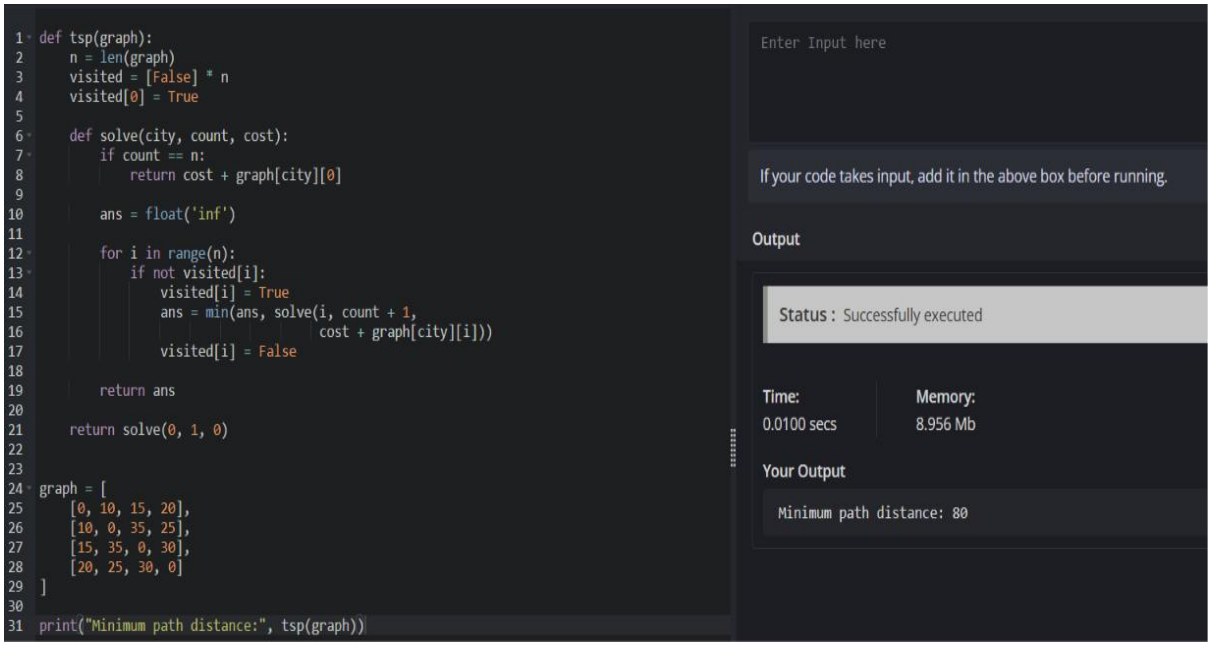
Time: 0.0000 secs

Memory: 9.052 Mb

Your Output

Minimum production time: 17

4. Write a c program to find the minimum path distance by using matrix form.



```python
def tsp(graph):
    n = len(graph)
    visited = [False] * n
    visited[0] = True

    def solve(city, count, cost):
        if count == n:
            return cost + graph[city][0]

        ans = float('inf')

        for i in range(n):
            if not visited[i]:
                visited[i] = True
                ans = min(ans, solve(i, count + 1,
                                     cost + graph[city][i]))
                visited[i] = False

        return ans

    return solve(0, 1, 0)


graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

print("Minimum path distance:", tsp(graph))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

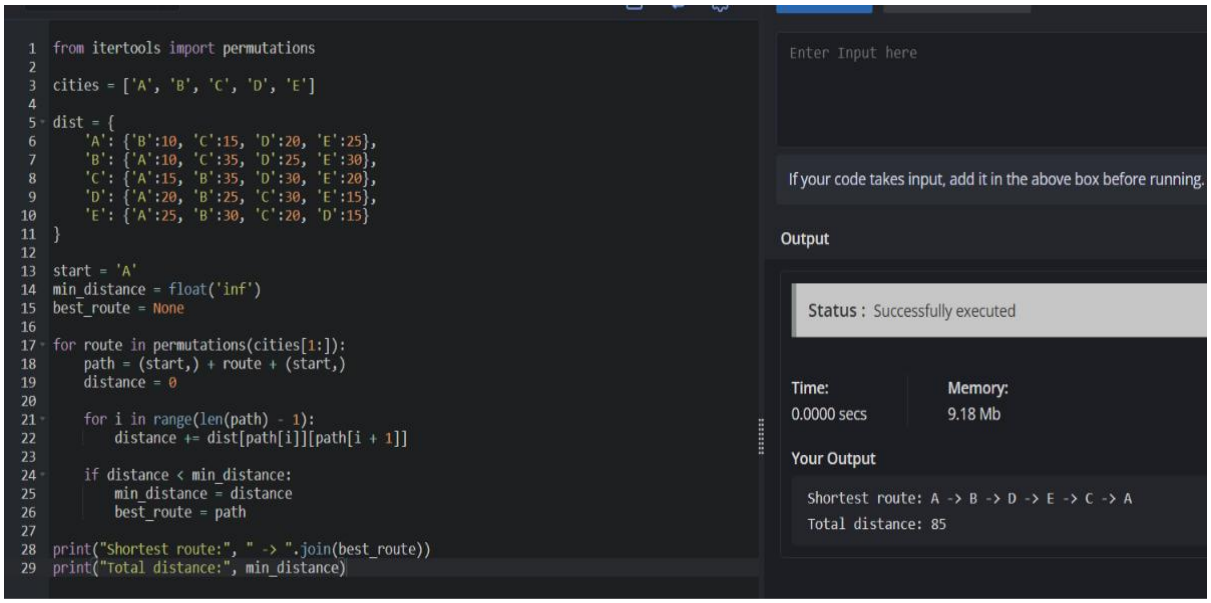
Time: 0.0100 secs

Memory: 8.956 Mb

Your Output

Minimum path distance: 80

5. Assume you are solving the Traveling Salesperson Problem for 4 cities (A, B, C, D) with known distances between each pair of cities. Now, you need to add a fifth city (E) to the problem.



```python
from itertools import permutations

cities = ['A', 'B', 'C', 'D', 'E']

dist = {
    'A': {'B':10, 'C':15, 'D':20, 'E':25},
    'B': {'A':10, 'C':35, 'D':25, 'E':30},
    'C': {'A':15, 'B':35, 'D':30, 'E':20},
    'D': {'A':20, 'B':25, 'C':30, 'E':15},
    'E': {'A':25, 'B':30, 'C':20, 'D':15}
}

start = 'A'
min_distance = float('inf')
best_route = None

for route in permutations(cities[1:]):
    path = (start,) + route + (start,)
    distance = 0

    for i in range(len(path) - 1):
        distance += dist[path[i]][path[i + 1]]

    if distance < min_distance:
        min_distance = distance
        best_route = path

print("Shortest route:", " -> ".join(best_route))
print("Total distance:", min_distance)
```

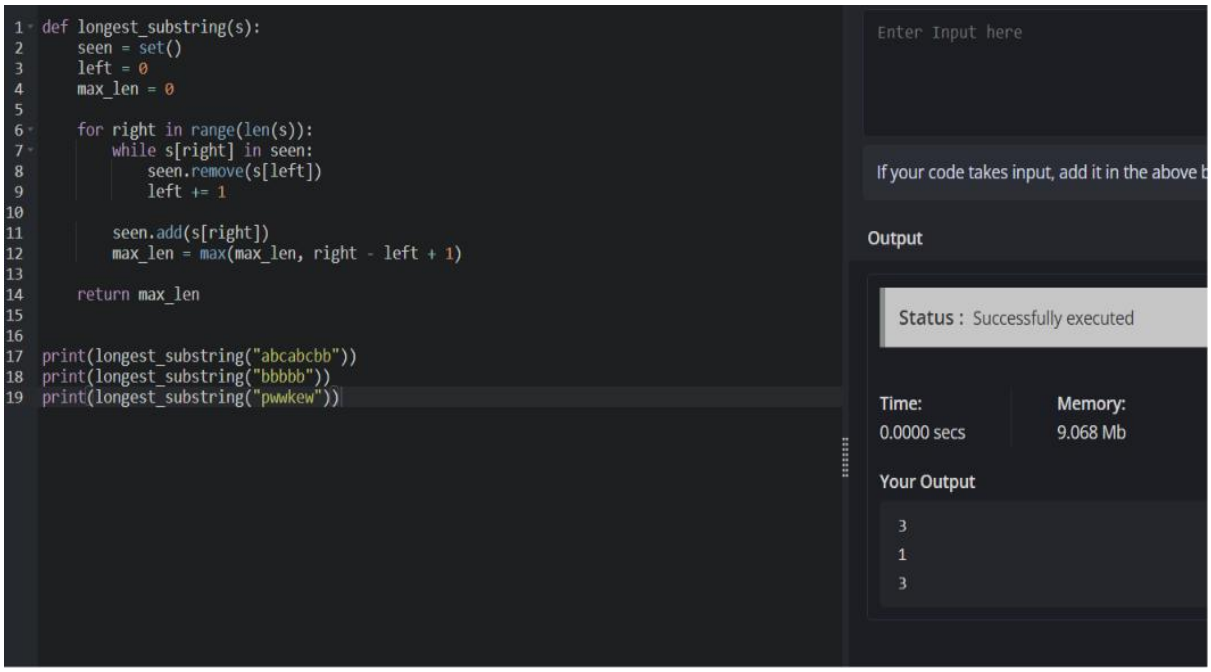
```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0000 secs

Memory:
9.18 Mb

Your Output

Shortest route: A -> B -> D -> E -> C -> A
Total distance: 85
```

6. Given a string s, return the longest palindromic substring in S.

```python
def longest_palindrome(s):
    start = 0
    max_len = 1

    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return left + 1, right - left - 1

    for i in range(len(s)):
        l1, len1 = expand(i, i)
        l2, len2 = expand(i, i + 1)

        if len1 > max_len:
            start = l1
            max_len = len1

        if len2 > max_len:
            start = l2
            max_len = len2

    return s[start:start + max_len]


print(longest_palindrome("babad"))
print(longest_palindrome("cbbd"))
```

```
Enter Input here

If your code takes input, add it in the abov

Output

Status : Successfully executed

Time:
0.0200 secs

Memory:
8.636 Mb

Your Output

bab
bb
```

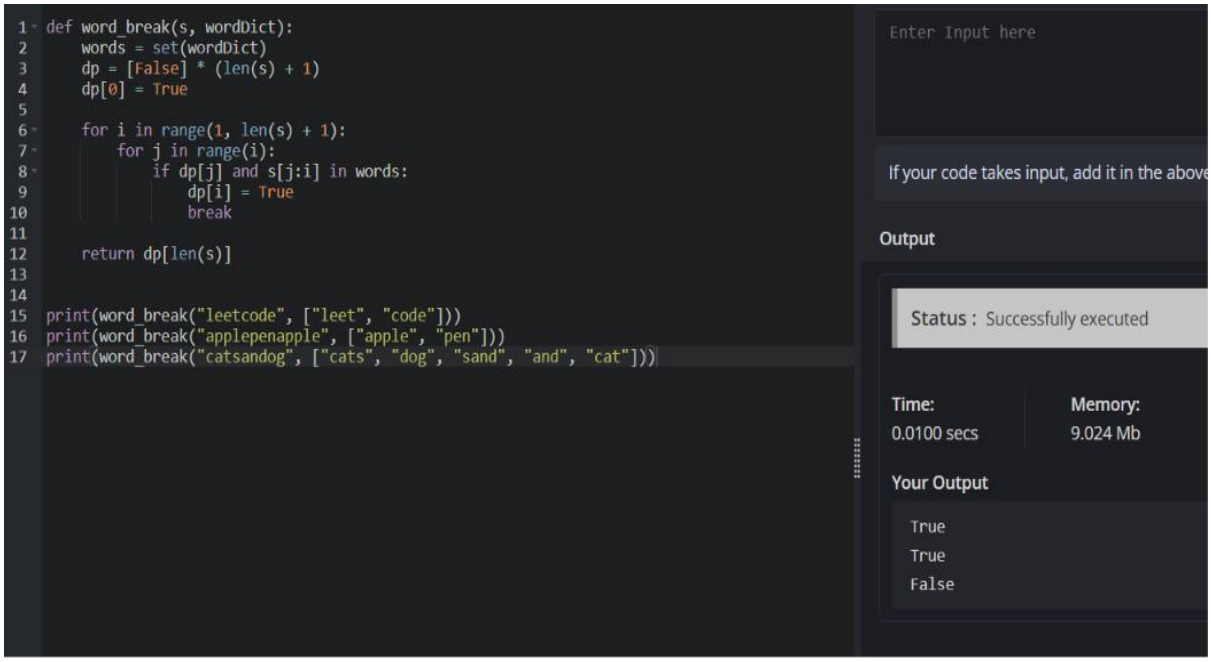
7. Given a string s, find the length of the longest substring without repeating characters.



```python
def longest_substring(s):
    seen = set()
    left = 0
    max_len = 0

    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1

        seen.add(s[right])
        max_len = max(max_len, right - left + 1)

    return max_len


print(longest_substring("abcabcbb"))
print(longest_substring("bbbbb"))
print(longest_substring("pwwkew"))
```

```
Enter Input here

If your code takes input, add it in the above b

Output

Status : Successfully executed

Time:
0.0000 secs

Memory:
9.068 Mb

Your Output

3
1
3
```

8. Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words. Note that the same word in the dictionary may be reused multiple times in the segmentation.



```python
def word_break(s, wordDict):
    words = set(wordDict)
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break

    return dp[len(s)]


print(word_break("leetcode", ["leet", "code"]))
print(word_break("applepenapple", ["apple", "pen"]))
print(word_break("catsandog", ["cats", "dog", "sand", "and", "cat"]))
```

```
Enter Input here

If your code takes input, add it in the above

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.024 Mb

Your Output

True
True
False
```

9. Given an input string and a dictionary of words, find out if the input string can be segmented into a space-separated sequence of dictionary words.Consider the following dictionary { i, like, sam, sung, samsung, mobile, ice, cream, icecream, man, go, mango}

```python
def word_break(s, dictionary):
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for word in dictionary:
            if i >= len(word) and dp[i - len(word)]:
                if s[i - len(word):i] == word:
                    dp[i] = True
                    break

    return dp[len(s)]


dictionary = {
    "i", "like", "sam", "sung", "samsung",
    "mobile", "ice", "cream", "icecream",
    "man", "go", "mango"
}

print("ilike:", "Yes" if word_break("ilike", dictionary) else "No")
print("ilikesamsung:", "Yes" if word_break("ilikesamsung", dictionary) else "No")
```

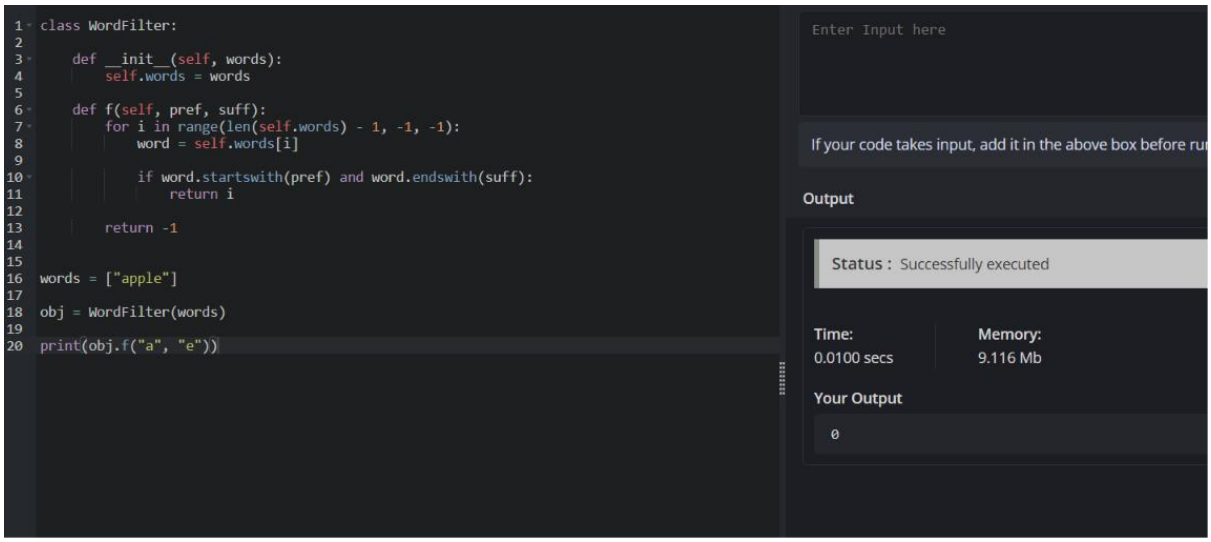
```
Enter Input here

If your code takes input, add it in the above box befor

Output

Status : Successfully executed

Time:
0.0200 secs

Memory:
9.056 Mb

Your Output

ilike: Yes
ilikesamsung: Yes
```

10. Given an array of strings words and a width maxWidth, format the text such that each line has exactly maxWidth characters and is fully (left and right) justified. You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly maxWidth characters. Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right. For the last line of text, it should be left- justified, and no extra space is inserted between words. A word is defined as a character sequence consisting of non-space characters only. Each word's length is guaranteed to be greater than 0 and not exceed maxWidth. The input array words contains at least one word.

```python
def justify(words, maxWidth):
    result = []
    i = 0

    while i < len(words):
        j = i
        length = 0

        while j < len(words) and length + len(words[j]) + (j - i) <= maxWidth:
            length += len(words[j])
            j += 1

        count = j - i
        spaces = maxWidth - length

        if j == len(words) or count == 1:
            line = " ".join(words[i:j])
            line += " " * (maxWidth - len(line))
        else:
            gap = spaces // (count - 1)
            extra = spaces % (count - 1)

            line = ""

            for k in range(count - 1):
                line += words[i + k]
                line += " " * (gap + (1 if k < extra else 0))

            line += words[j - 1]

        result.append(line)
```

```
Enter Input here

If your code takes input, add it in the above box befor

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.036 Mb

Your Output

This is an
example of text
justification.
```

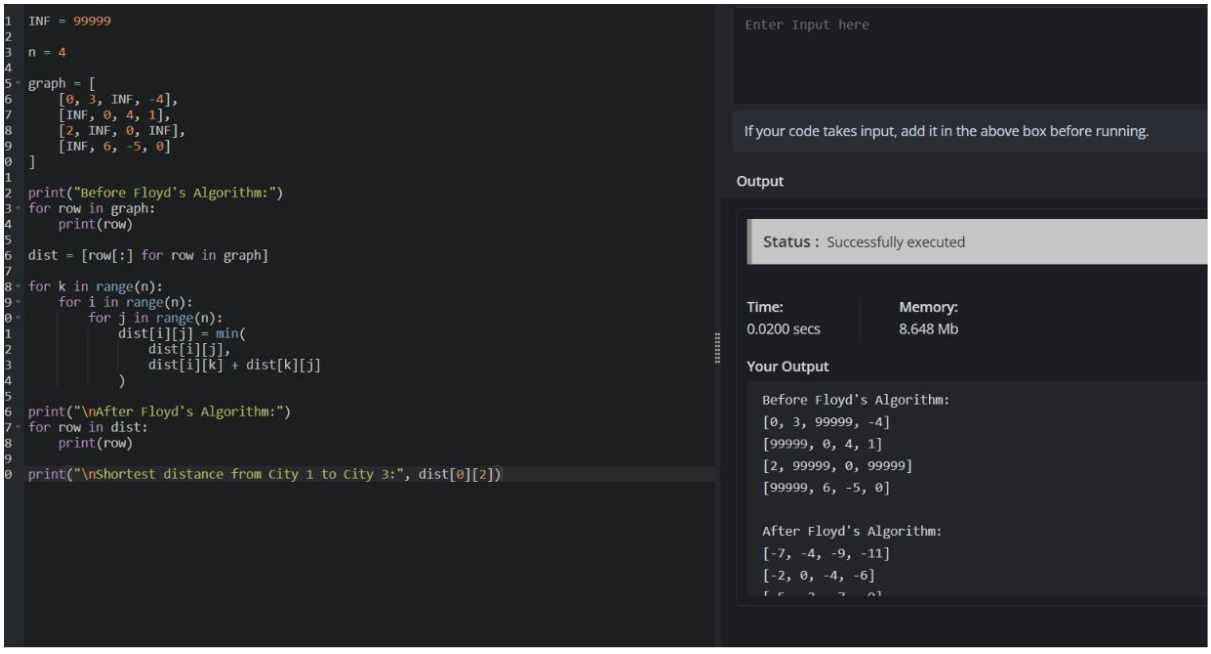
11. Design a special dictionary that searches the words in it by a prefix and a suffix. Implement the WordFilter class: WordFilter(string[] words) Initializes the object with the words in the dictionary.f(string pref, string suff) Returns the index of the word in the dictionary, which has the prefix pref and the suffix suff. If there is more than one valid index, return the largest of them. If there is no such word in the dictionary, return -1.



```
class WordFilter:

    def __init__(self, words):
        self.words = words

    def f(self, pref, suff):
        for i in range(len(self.words) - 1, -1, -1):
            word = self.words[i]

            if word.startswith(pref) and word.endswith(suff):
                return i

        return -1


words = ["apple"]

obj = WordFilter(words)

print(obj.f("a", "e"))
```

```
Enter Input here

If your code takes input, add it in the above box before ru

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.116 Mb

Your Output

0
```

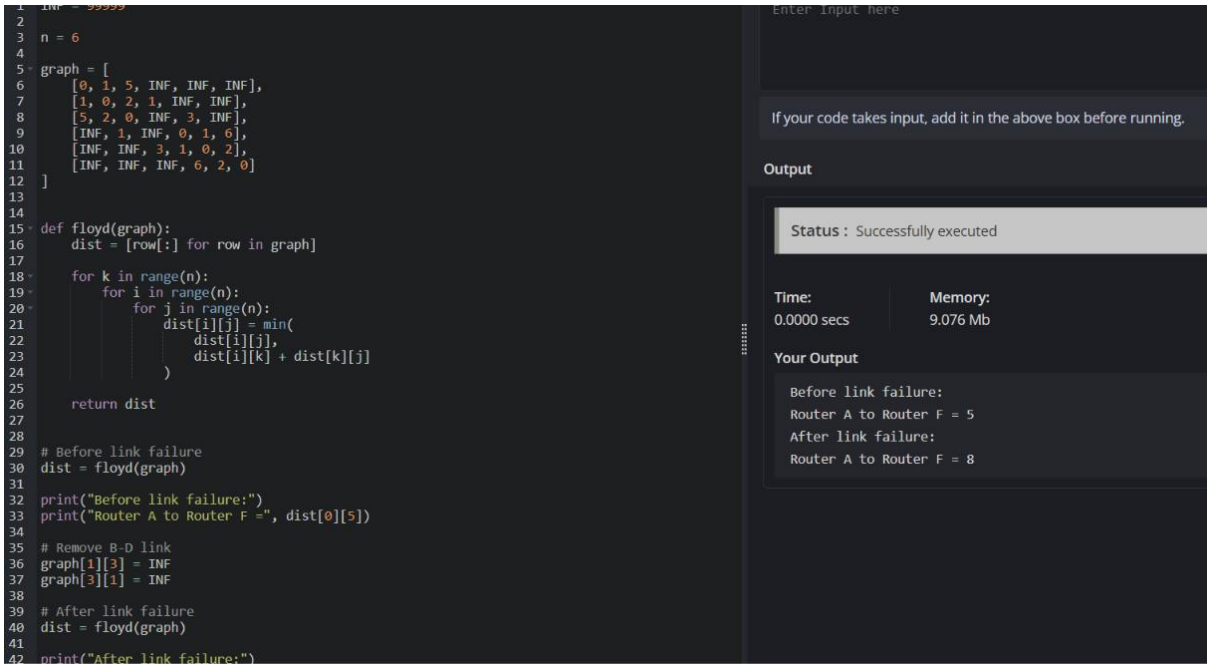
12. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path



```
INF = 99999

n = 4

graph = [
    [0, 3, INF, -4],
    [INF, 0, 4, 1],
    [2, INF, 0, INF],
    [INF, 6, -5, 0]
]

print("Before Floyd's Algorithm:")
for row in graph:
    print(row)

dist = [row[:] for row in graph]

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(
                dist[i][j],
                dist[i][k] + dist[k][j]
            )

print("\nAfter Floyd's Algorithm:")
for row in dist:
    print(row)

print("\nShortest distance from City 1 to City 3:", dist[0][2])
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0200 secs

Memory:
8.648 Mb

Your Output

Before Floyd's Algorithm:
[0, 3, 99999, -4]
[99999, 0, 4, 1]
[2, 99999, 0, 99999]
[99999, 6, -5, 0]

After Floyd's Algorithm:
[-7, -4, -9, -11]
[-2, 0, -4, -6]
```

13. Write a Program to implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a change where the link between Router B and Router D fails. Update the distance matrix accordingly. Display the shortest path from Router A to Router F before and after the link failure.



```
INF = 99999

n = 6

graph = [
    [0, 1, 5, INF, INF, INF],
    [1, 0, 2, 1, INF, INF],
    [5, 2, 0, INF, 3, INF],
    [INF, 1, INF, 0, 1, 6],
    [INF, INF, 3, 1, 0, 2],
    [INF, INF, INF, 6, 2, 0]
]


def floyd(graph):
    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    return dist


# Before link failure
dist = floyd(graph)

print("Before link failure:")
print("Router A to Router F =", dist[0][5])

# Remove B-D link
graph[1][3] = INF
graph[3][1] = INF

# After link failure
dist = floyd(graph)

print("After link failure:")
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

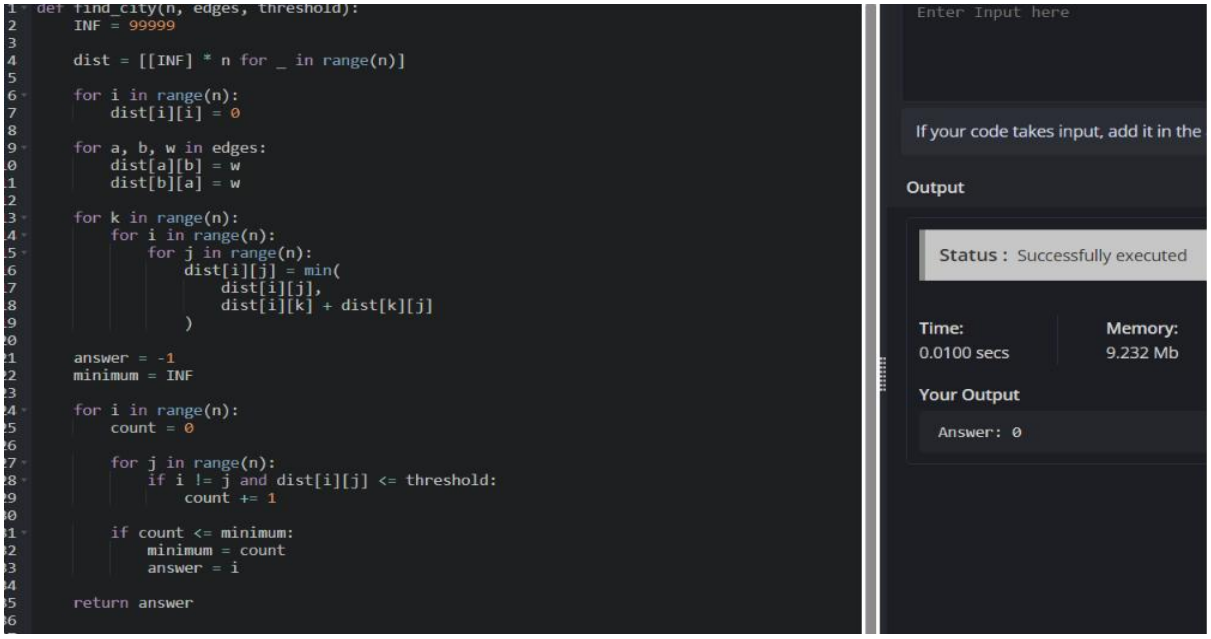
Time: 0.0000 secs

Memory: 9.076 Mb

Your Output

```
Before link failure:
Router A to Router F = 5
After link failure:
Router A to Router F = 8
```

14. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path



```
def find_city(n, edges, threshold):
    INF = 99999

    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for a, b, w in edges:
        dist[a][b] = w
        dist[b][a] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    answer = -1
    minimum = INF

    for i in range(n):
        count = 0

        for j in range(n):
            if i != j and dist[i][j] <= threshold:
                count += 1

        if count <= minimum:
            minimum = count
            answer = i

    return answer
```

Enter Input here

If your code takes input, add it in the

Output

Status : Successfully executed

Time: 0.0100 secs

Memory: 9.232 Mb

Your Output

```
Answer: 0
```

15. Implement the Optimal Binary Search Tree algorithm for the keys A,B,C,D with frequencies 0.1,0.2,0.4,0.3 Write the code using any programming language to construct the OBST for the given keys and frequencies. Execute your code and display the resulting OBST and its cost. Print the cost and root matrix.

```python
def optimal_bst(keys, freq):
    n = len(keys)

    cost = [[0] * n for _ in range(n)]
    root = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]
        root[i][i] = i

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            total = sum(freq[i:j + 1])
            cost[i][j] = float('inf')

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0

                value = left + right + total

                if value < cost[i][j]:
                    cost[i][j] = value
                    root[i][j] = r

    print("Cost:", cost[0][n - 1])

    print("\nCost Matrix:")
    for row in cost:
        print(row)

    print("\nRoot Matrix:")
    for row in root:
        print(row)

    return cost, root
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0000 secs

Memory:
9.344 Mb

Your Output

Cost Matrix:
[0.1, 0.4, 1.1, 1.7]
[0, 0.2, 0.8, 1.4]
[0, 0, 0.4, 1.0]
[0, 0, 0, 0.3]

Root Matrix:
[0, 1, 2, 2]
[0, 1, 2, 2]
```

16. Consider a set of keys 10,12,16,21 with frequencies 4,2,6,3 and the respective probabilities. Write a Program to construct an OBST in a programming language of your choice. Execute your code and display the resulting OBST, its cost and root matrix.

```python
    n = len(keys)

    cost = [[0] * n for _ in range(n)]
    root = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]
        root[i][i] = i

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            total = sum(freq[i:j + 1])
            cost[i][j] = float('inf')

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0

                value = left + right + total

                if value < cost[i][j]:
                    cost[i][j] = value
                    root[i][j] = r

    print("Minimum Cost:", cost[0][n - 1])

    print("\nCost Matrix:")
    for row in cost:
        print(row)

    print("\nRoot Matrix:")
    for row in root:
        print(row)


keys = [10, 12, 16, 21]
freq = [4, 2, 6, 3]
```

```
If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.16 Mb

Your Output

Minimum Cost: 26

Cost Matrix:
[4, 8, 20, 26]
[0, 2, 10, 16]
[0, 0, 6, 12]
[0, 0, 0, 3]

Root Matrix:
```

17. A game on an undirected graph is played by two players, Mouse and Cat, who alternate turns. The graph is given as follows: graph[a] is a list of all nodes bsuch that ab is an edge of the graph. The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and there is a hole at node 0. During each player's turn, they must travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it must travel to anynode in graph[1]. Additionally, it is not allowed for the Cat to travel to the Hole (node 0).Then, the game can end in three ways:
    - If ever the Cat occupies the same node as the Mouse, the Cat wins.
    - If ever the Mouse reaches the Hole, the Mouse wins.
    - If ever a position is repeated (i.e., the players are in the same position as a
    - previous turn, and it is the same player's turn to move), the game is a draw.

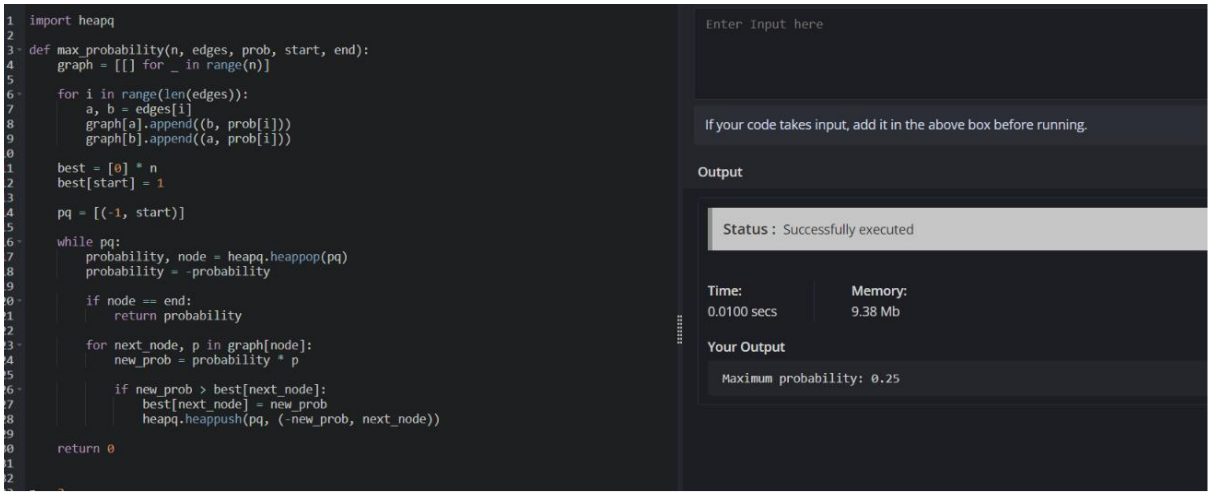
    Given a graph, and assuming both players play optimally, return

    - 1 if the mouse wins the game,
    - 2 if the cat wins the game, or
    - 0 if the game is a draw.

```
        q.append((0, c, 1))

    # Cat catches mouse -> Cat wins
    for i in range(1, n):
        result[i][i][0] = 2
        result[i][i][1] = 2
        q.append((i, i, 0))
        q.append((i, i, 1))

    while q:
        m, c, turn = q.popleft()
        res = result[m][c][turn]

        if turn == 0:
            # Previous move was by Cat
            parents = [(m, pc, 1) for pc in graph[c] if pc != 0]
        else:
            # Previous move was by Mouse
            parents = [(pm, c, 0) for pm in graph[m]]

        for pm, pc, pt in parents:
            if result[pm][pc][pt] != 0:
                continue

            if (pt == 0 and res == 1) or (pt == 1 and res == 2):
                result[pm][pc][pt] = res
                q.append((pm, pc, pt))
            else:
                degree[pm][pc][pt] -= 1

                if degree[pm][pc][pt] == 0:
                    result[pm][pc][pt] = res
                    q.append((pm, pc, pt))

    return result[1][2][0]
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.724 Mb

Your Output

Result: 0
```

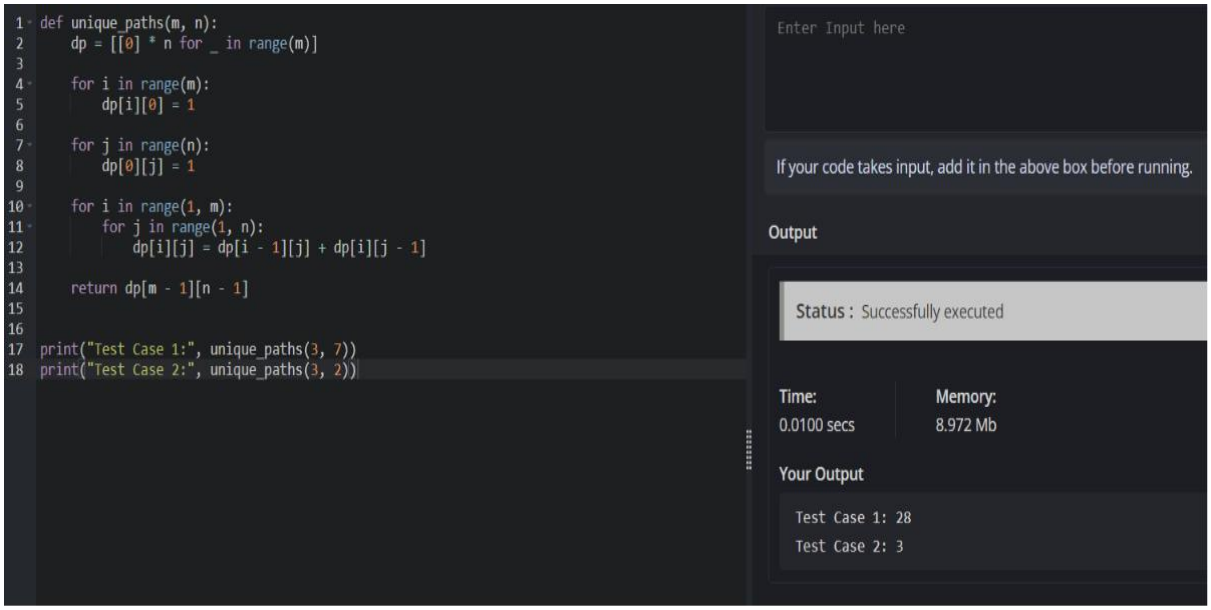
18. You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where edges[i] = [a, b] is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge succProb[i]. Given two nodes start and end, find the path with the maximum probability of success to go from start to end and return its success probability. If there is no path from start to end, return 0. Your answer will be accepted if it differs from the correct answer by at most 1e-5.



```python
import heapq

def max_probability(n, edges, prob, start, end):
    graph = [[] for _ in range(n)]

    for i in range(len(edges)):
        a, b = edges[i]
        graph[a].append((b, prob[i]))
        graph[b].append((a, prob[i]))

    best = [0] * n
    best[start] = 1

    pq = [(-1, start)]

    while pq:
        probability, node = heapq.heappop(pq)
        probability = -probability

        if node == end:
            return probability

        for next_node, p in graph[node]:
            new_prob = probability * p

            if new_prob > best[next_node]:
                best[next_node] = new_prob
                heapq.heappush(pq, (-new_prob, next_node))

    return 0
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.38 Mb

Your Output

Maximum probability: 0.25
```

19. There is a robot on an m x n grid. The robot is initially located at the top-left corner (i.e., grid[0][0]). The robot tries to move to the bottom-right corner (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any point in time. Given the two integers m and n, return the number of possible unique paths that the robot can take to reach the bottom-right corner. The test cases are generated so that the answer will be less than or equal to 2 * 10 9.



```python
def unique_paths(m, n):
    dp = [[0] * n for _ in range(m)]

    for i in range(m):
        dp[i][0] = 1

    for j in range(n):
        dp[0][j] = 1

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

    return dp[m - 1][n - 1]


print("Test Case 1:", unique_paths(3, 7))
print("Test Case 2:", unique_paths(3, 2))
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.972 Mb

Your Output

Test Case 1: 28
Test Case 2: 3
```

20. Given an array of integers nums, return the number of good pairs. A pair (i, j) is called good if nums[i] == nums[j] and i < j.

```python
def good_pairs(nums):
    count = 0

    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j]:
                count += 1

    return count


print(good_pairs([1, 2, 3, 1, 1, 3]))
print(good_pairs([1, 1, 1, 1]))
```

```
Enter Input here

If your code takes input, add it in the above box b

Output

Status : Successfully executed

Time:
0.0000 secs

Memory:
8.992 Mb

Your Output

4
6
```

21. There are n cities numbered from 0 to n-1. Given the array edges where edges[i] = [fromi, toi, weighti] represents a bidirectional and weighted edge between cities fromi and toi, and given the integer distanceThreshold. Return the city with the smallest number of cities that are reachable through some path and whose distance is at most distanceThreshold, If there are multiple such cities, return the city with the greatest number. Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

```python
    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for a, b, w in edges:
        dist[a][b] = w
        dist[b][a] = w

    # Floyd-Warshall Algorithm
    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    answer = -1
    minimum = INF

    for i in range(n):
        count = 0

        for j in range(n):
            if i != j and dist[i][j] <= threshold:
                count += 1

        if count <= minimum:
            minimum = count
            answer = i
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.196 Mb

Your Output

city: 3
```

22. You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges times[i] = (ui, vi, wi), where ui is the source node, vi is the target node, and wi is the time it takes for a signal to travel from source to target. We will send a signal from a given node k. Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

```python
def network_delay_time(times, n, k):
    graph = [[] for _ in range(n + 1)]

    for u, v, w in times:
        graph[u].append((v, w))

    distance = [float('inf')] * (n + 1)
    distance[k] = 0

    pq = [(0, k)]

    while pq:
        time, node = heapq.heappop(pq)

        if time > distance[node]:
            continue

        for next_node, weight in graph[node]:
            new_time = time + weight

            if new_time < distance[next_node]:
                distance[next_node] = new_time
                heapq.heappush(pq, (new_time, next_node))

    answer = max(distance[1:])

    if answer == float('inf'):
        return -1

    return answer


print(network_delay_time(
    [[2, 1, 1], [2, 3, 1], [3, 4, 1]],
    4,
    2
))
```

Enter Input here

If your code takes input, add it in the abov

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.116 Mb

Your Output

```
2
1
-1
```